

移动应用开发实验

封面信息

项目	内容
实验名称	微信小程序开发
实验编号	d20301035104、d20301009204
学号	2023210626
姓名	王建
班级	软件工程2班
日期	2026-04-24
组别	第2组
项目名称	智慧校园通知小程序
技术栈	微信小程序 (JavaScript)

实验目的

- 掌握微信小程序开发流程
- 学会WXML/WXSS/JS开发
- 理解页面路由与数据传递
- 掌握本地存储使用
- 体验AI辅助开发

实验环境

工具	版本	用途
微信开发者工具	最新版	小程序开发与调试
Node.js	18+	可选，用于npm依赖管理
AI辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 页面流程图

首页(通知列表) → 详情页(通知详情)
↓
本地存储(已读状态)

1.2 数据模型

```
// 通知数据结构
{
  id: 1,
  title: "关于期末考试安排的通知",
  content: "详细内容...",
  author: "教务处",
  time: "2024-01-01",
  isRead: false
}
```

1.3 系统架构图

智慧校园通知小程序的系统架构如图1所示，采用经典的MVC三层架构设计，各层职责明确，相互协作完成用户需求。

- **用户层(User)**：最顶层，代表小程序的最终用户。用户通过与界面交互来发起操作，比如点击查看通知列表、查看通知详情等。用户层只负责展示和输入，不处理任何业务逻辑。
- **视图层(View)**：由WXML和WXSS组成，负责页面渲染和展示。
 - **通知列表页面(pages/index/index)**：展示所有校园通知的列表界面。每个通知项显示标题、发布部门和发布时间。已读通知通过半透明效果视觉区分。
 - **通知详情页面(pages/detail/detail)**：展示单条通知的详细内容界面，包括完整通知内容、发布部门、发布时间等信息。
视图层通过 `bindtap` 等事件绑定响应用户操作，将事件传递给逻辑层处理。
- **逻辑层(Logic)**：使用JavaScript实现，负责核心业务逻辑处理。
 - **IndexPage逻辑**：主要负责通知列表页面的业务逻辑
 - `loadNews()`：加载通知数据，包括模拟数据和从本地存储读取已读状态
 - `goToDetail()`：处理用户点击通知项事件，跳转到详情页
 - `onPullDownRefresh()`：处理下拉刷新事件，重新加载数据
 - **DetailPage逻辑**：主要负责通知详情页面的业务逻辑
 - `loadDetail(id)`：根据通知ID查找并加载对应通知详情
 - `markAsRead(id)`：将通知标记为已读，并保存到本地存储逻辑层通过 `setData()` 方法将数据更新传递给视图层，实现界面刷新。
- **数据层(Data)**：使用微信小程序的本地存储API(`wx.storage`)实现数据持久化。
 - 存储已读通知的ID数组，键名为'`readIds`'
 - 提供 `getStorageSync(key)` 读取数据和 `setStorageSync(key, data)` 写入数据两个同步方法
 - 数据持久化保证用户关闭小程序后再次打开时，已读状态仍然保留

数据流向说明：

1. 用户触发视图层事件 → 视图层调用逻辑层方法
2. 逻辑层处理业务逻辑 → 读写数据层
3. 数据层返回结果 → 逻辑层更新数据
4. 逻辑层通过`setData()` → 视图层重新渲染

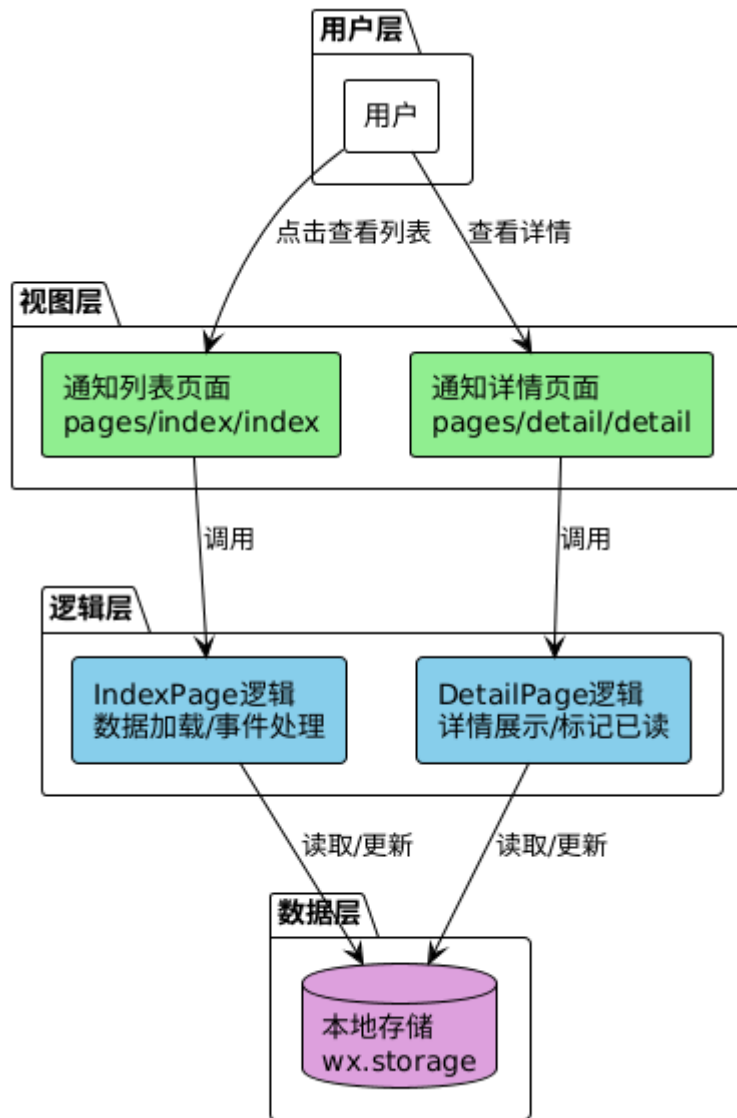


图 1 系统架构图

1.4 类图

图2展示了智慧校园通知小程序的核心类结构，清晰描述了各个类的属性、方法以及它们之间的关系。

核心类说明：

- **Page类**：微信小程序页面基类，所有页面都继承自该类
 - 属性：
 - `data: Object`：页面数据对象，存储页面的状态数据，通过`setData()`更新
 - 方法：
 - `onLoad()`：页面生命周期函数，页面加载时自动调用
 - `onShow()`：页面生命周期函数，页面显示时自动调用
 - `onPullDownRefresh()`：下拉刷新触发时调用，需在配置中开启
 - `setData()`：更新页面数据，触发视图重新渲染
- **IndexPage类**：继承自Page，通知列表页面的具体实现
 - 属性：
 - `newsList: Array<News>`：通知列表数组，存储所有通知对象
 - 方法：

- `onLoad()`：页面加载时调用，调用`loadNews()`初始化数据
 - `loadNews()`：加载通知数据，创建模拟通知数组，从`Storage`读取已读状态，合并后调用`setData()`更新列表
 - `goToDetail()`：用户点击通知项时调用，通过 `data-id` 获取通知ID，使用 `wx.navigateTo()` 跳转到详情页
 - `onPullDownRefresh()`：下拉刷新时调用，重新执行`loadNews()`，完成后调用 `wx.stopPullDownRefresh()` 停止刷新动画
- **DetailPage类**：继承自`Page`，通知详情页面的具体实现
 - 属性：
 - `news: News`：当前展示的单条通知对象
 - 方法：
 - `onLoad(options)`：页面加载时调用，从`options`参数中获取通知ID，调用 `loadDetail(id)`
 - `loadDetail(id)`：根据ID在模拟数据数组中查找对应的通知，调用`setData()`更新页面显示
 - `markAsRead(id)`：将通知标记为已读，先从`Storage`读取现有已读ID数组，如果当前ID不在数组中则添加，然后写回`Storage`
- **News类**：通知数据模型类，封装通知信息
 - 属性：
 - `id: Integer`：通知唯一标识符
 - `title: String`：通知标题
 - `content: String`：通知详细内容
 - `author: String`：发布部门，如"教务处"、"图书馆"
 - `time: String`：发布时间，格式"YYYY-MM-DD"
 - `isRead: Boolean`：已读状态标记，`true`表示已读，`false`表示未读
- **Storage类**：本地存储工具类，封装微信小程序的存储API
 - 方法：
 - `setStorageSync(key, data)`：同步写入数据，接受键名和数据两个参数
 - `getStorageSync(key)`：同步读取数据，根据键名返回对应数据

类关系说明：

- **继承关系**：`IndexPage`和`DetailPage`都继承自`Page`，拥有`Page`的所有属性和方法
- **组合关系**：`IndexPage`包含`News`数组(1对多)，`DetailPage`包含单个`News`对象(1对1)
- **依赖关系**：`IndexPage`和`DetailPage`都依赖`Storage`来读写已读状态数据

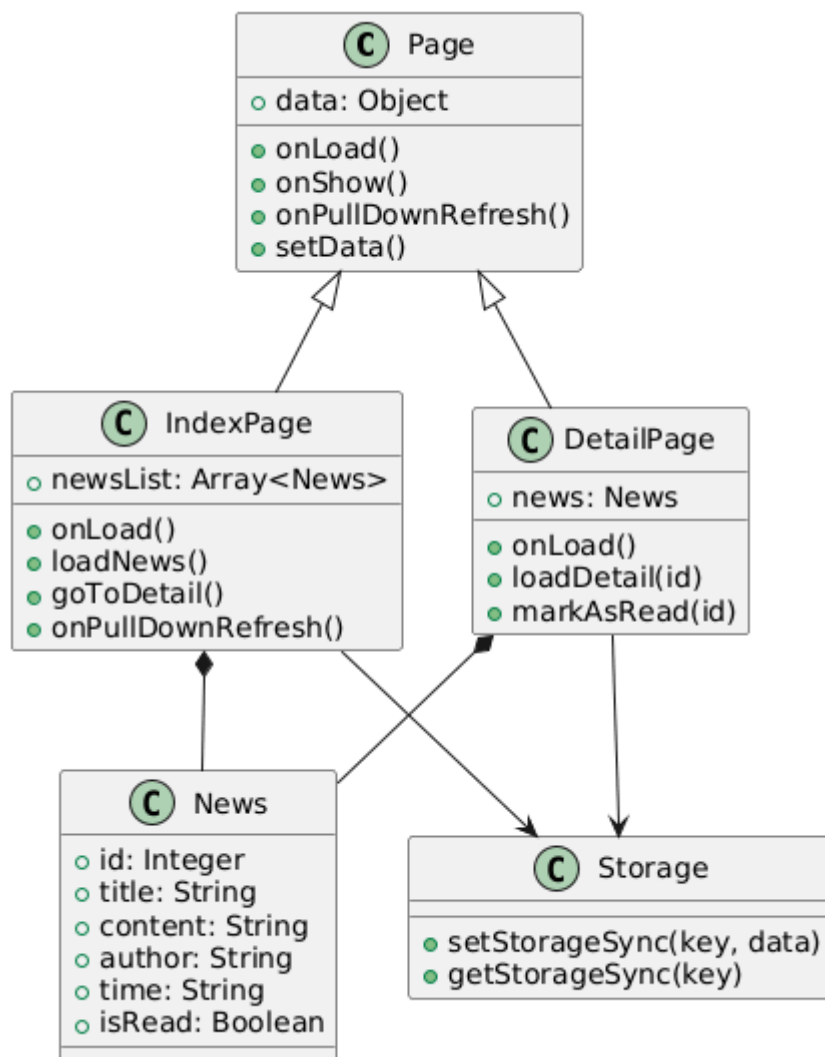


图 2 系统类图

1.5 顺序图

图3展示了用户查看通知详情的完整交互流程，详细描述了从打开小程序到查看完通知的所有步骤和对象间的调用关系。

第一步：加载通知列表（第1步-第7步）

1. **用户打开小程序**：用户在微信中点击小程序图标，小程序启动并进入首页
2. **IndexPage触发onLoad生命周期**：通知列表页面加载，自动调用onLoad()方法
3. **onLoad调用loadNews()**：页面加载后立即调用loadNews()方法初始化列表数据
4. **从Storage读取已读状态**：loadNews()内部调用 `wx.getStorageSync('readIds')` 从本地存储获取已读通知的ID数组
5. **Storage返回已读ID数组**：如果用户之前没有读过通知，返回空数组[]，否则返回包含已读ID的数组
6. **构建newsList数据**：loadNews()创建模拟通知数据数组，遍历每个通知，根据已读ID数组设置isRead标记
7. **更新页面显示**：调用 `this.setData({newsList})`，将数据从逻辑层传递到视图层，触发WXML重新渲染，显示通知列表

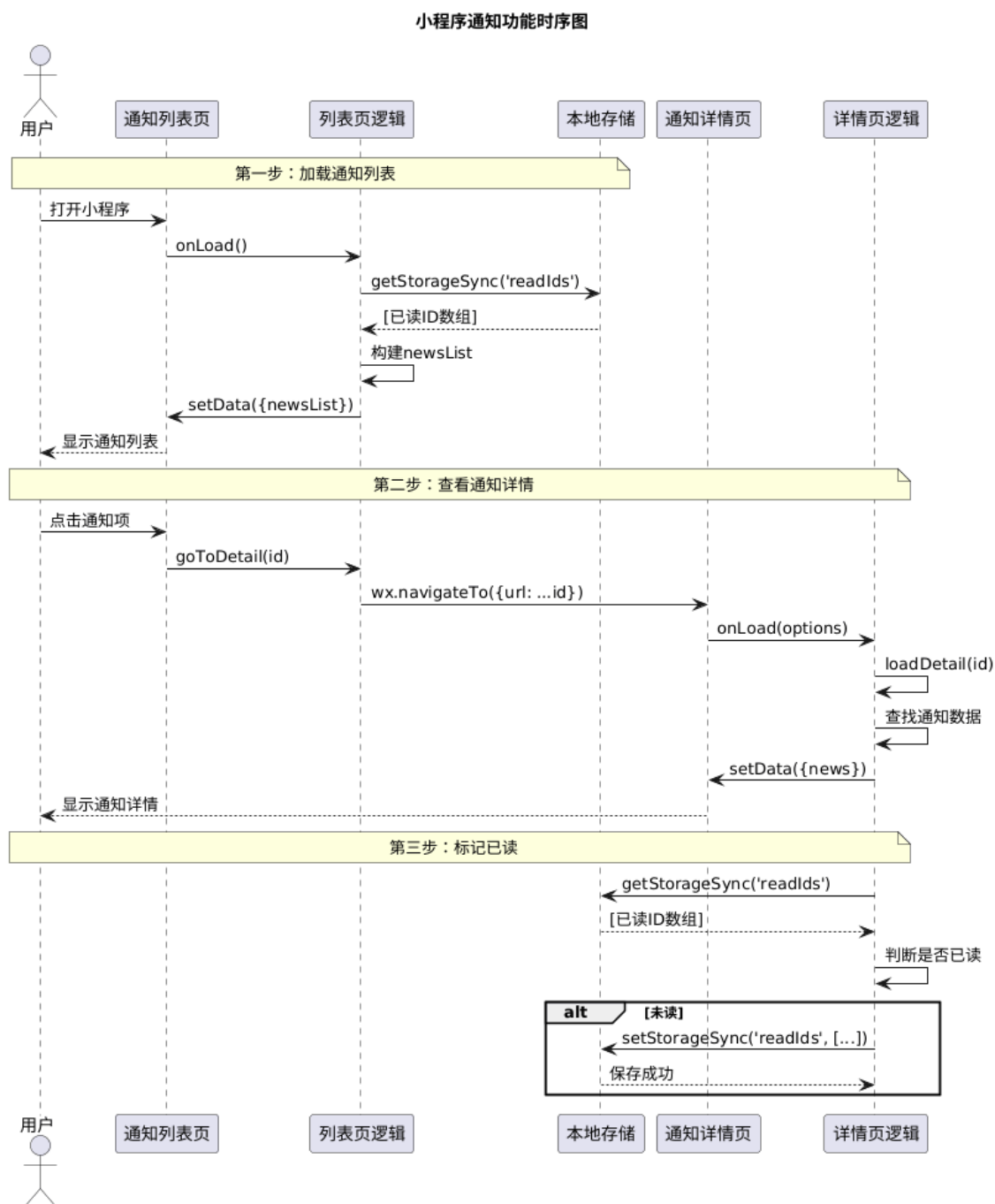


图 3 查看通知详情顺序图

第二步：查看通知详情（第8步-第14步）

8. **用户点击通知项**：用户在列表中点击感兴趣的通知
9. **IndexPage触发goToDetail事件**：点击事件通过bindtap绑定触发goToDetail方法，通过event.currentTarget.dataset.id获取通知ID
10. **调用wx.navigateTo跳转**：goToDetail方法调用微信小程序API wx.navigateTo()，传递URL参数如'/pages/detail/detail?id=1'
11. **DetailPage触发onLoad生命周期**：详情页面加载，onLoad方法接收options参数，从中解析出通知ID
12. **onLoad调用loadDetail(id)**：将ID传递给loadDetail方法，加载具体通知内容
13. **查找通知数据**：loadDetail在模拟数据数组中通过id查找匹配的通知对象
14. **更新详情页显示**：调用 `this.setData({news})`，将通知对象数据传递给视图层，页面显示通知详情

第三步：标记已读状态（第15步-第19步）

15. **从Storage读取已读ID**：loadDetail中调用markAsRead(id)，首先从Storage读取现有的已读ID数组
16. **Storage返回已读ID数组**：获取当前所有已读通知的ID列表
17. **判断是否已读**：检查当前通知ID是否在已读数组中
18. **如果未读，添加到已读数组**：如果不在数组中，使用Array.push()将当前ID添加到数组末尾
19. **保存到Storage**：调用 `wx.setStorageSync('readIds', updatedArray)` 将更新后的已读ID数组写回本地存储，实现数据持久化

关键调用机制说明：

- **数据绑定**：视图层通过`{{变量名}}`语法绑定逻辑层数据，setData触发视图更新
- **事件传递**：通过bindtap绑定事件，通过data-*属性传递自定义数据
- **页面跳转**：wx.navigateTo()保留当前页面，压入页面栈，可通过返回键返回
- **本地存储**：使用同步API保证数据一致性，避免异步回调问题

2. 创建小程序项目

2.1 新建项目

- ☒ 打开微信开发者工具
- ☒ 新建小程序项目
- ☒ 填写AppID（使用测试号）
- ☒ 选择JavaScript模板

3. 通知列表页面开发

3.1 WXML布局

```
<!-- pages/index/index.wxml -->
<view class="container">
  <view class="news-list">
    <view
      class="news-item {{item.isRead ? 'read' : ''}}"
      wx:for="{{newsList}}"
      wx:key="id"
      bindtap="goToDetail"
      data-id="{{item.id}}"
    >
      <view class="title">{{item.title}}</view>
      <view class="info">
        <text>{{item.author}}</text>
        <text>{{item.time}}</text>
      </view>
    </view>
  </view>
</view>
```

3.2 WXSS样式

```
/* pages/index/index.wxss */
.news-item {
  padding: 16px;
  border-bottom: 1px solid #eee;
  background: #fff;
}
.news-item.read {
  opacity: 0.6;
}
.title {
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 8px;
}
.info {
  font-size: 12px;
  color: #999;
  display: flex;
  justify-content: space-between;
}
```

3.3 JS逻辑

```
// pages/index/index.js
Page({
  data: {
    newsList: []
  },

  onLoad() {
    this.loadNews();
  },

  onPullDownRefresh() {
    this.loadNews().then(() => {
      wx.stopPullDownRefresh();
    });
  },

  loadNews() {
    // 模拟数据
    const newsList = [
      { id: 1, title: "关于期末考试安排的通知", content: "详细内容...", author: "教务处", time: "2024-01-01", isRead: false },
      { id: 2, title: "图书馆开放时间调整", content: "详细内容...", author: "图书馆", time: "2024-01-02", isRead: false },
      { id: 3, title: "校园网络维护通知", content: "详细内容...", author: "信息中心", time: "2024-01-03", isRead: false }
    ];

    // 读取本地存储的已读状态
    const readIds = wx.getStorageSync('readIds') || [];
  }
});
```



```

newsList.forEach(news => {
  news.isRead = readIds.includes(news.id);
});

this.setData({ newsList });
},

goToDetail(e) {
  const id = e.currentTarget.dataset.id;
  wx.navigateTo({
    url: `/pages/detail/detail?id=${id}`
  });
}
});

```

3.4 配置文件

```

// pages/index/index.json
{
  "navigationBarTitleText": "智慧校园通知",
  "enablePullDownRefresh": true,
  "usingComponents": {}
}

```

4. 详情页开发

4.1 WXML布局

```

<!-- pages/detail/detail.wxml -->
<view class="container">
  <view class="title">{{news.title}}</view>
  <view class="info">
    <text>发布部门: {{news.author}}</text>
    <text>发布时间: {{news.time}}</text>
  </view>
  <view class="content">{{news.content}}</view>
</view>

```

4.2 WXSS样式

```

/* pages/detail/detail.wxss */
.container {
  padding: 20px;
}

.title {
  font-size: 20px;
  font-weight: bold;
  margin-bottom: 16px;
  color: #333;
}

.info {

```

```

display: flex;
justify-content: space-between;
padding: 12px 0;
border-top: 1px solid #eee;
border-bottom: 1px solid #eee;
margin-bottom: 16px;
color: #666;
font-size: 14px;
}

.content {
font-size: 15px;
line-height: 1.8;
color: #444;
}

```

4.3 JS逻辑

```

// pages/detail/detail.js
Page({
  data: {
    news: {}
  },

  onLoad(options) {
    const id = parseInt(options.id);
    this.loadDetail(id);
  },

  loadDetail(id) {
    // 模拟数据
    const allNews = [
      { id: 1, title: "关于期末考试安排的通知", content: "本次期末考试安排如下：\n\n1. 考试时间：2024年1月15日-1月20日\n2. 考试地点：各教学楼\n3. 注意事项：请携带学生证和身份证\n\n请同学们按时参加考试，祝大家取得好成绩！", author: "教务处", time: "2024-01-01" },
      { id: 2, title: "图书馆开放时间调整", content: "图书馆开放时间调整如下：\n\n周一至周五：8:00-22:00\n周六周日：9:00-21:00\n\n请同学们合理安排学习时间。", author: "图书馆", time: "2024-01-02" },
      { id: 3, title: "校园网络维护通知", content: "为提升校园网络质量，信息中心将于1月5日进行网络维护。\n\n维护时间：1月5日 22:00-次日6:00\n影响范围：全校网络\n\n请同学们提前做好准备，给您带来的不便敬请谅解。", author: "信息中心", time: "2024-01-03" }
    ];

    const news = allNews.find(n => n.id === id);
    this.setData({ news });

    // 标记为已读
    this.markAsRead(id);
  },

  markAsRead(id) {
    let readIds = wx.getStorageSync('readIds') || [];
    if (!readIds.includes(id)) {
      readIds.push(id);
      wx.setStorageSync('readIds', readIds);
    }
  }
});

```

```
    }  
  }  
});
```

4.4 配置文件

```
// pages/detail/detail.json  
{  
  "navigationBarTitleText": "通知详情",  
  "usingComponents": {}  
}
```

5. 本地存储功能

5.1 已读状态存储

```
// 保存已读ID  
wx.setStorageSync('readIds', [1, 2, 3]);  
  
// 读取已读ID  
const readIds = wx.getStorageSync('readIds');  
  
// 检查是否已读  
const isRead = readIds.includes(newsId);
```

6. 测试与发布

6.1 真机预览

- ☒ 编译项目
- ☒ 扫描二维码预览
- ☒ 测试各项功能

6.2 体验版发布

- ☒ 点击上传
- ☒ 填写版本信息
- ☒ 获取体验版二维码

实验结果

验证结果

功能	状态	备注
通知列表展示	[✓] 成功	列表正常渲染，已读状态显示正常
详情页跳转	[✓] 成功	点击列表项可跳转到详情页并显示对应内容
已读状态存储	[✓] 成功	本地存储功能正常，刷新后状态保持
下拉刷新	[✓] 成功	下拉可刷新列表数据

功能	状态	备注
真机预览	[✓] 成功	小程序在真机上正常运行

截图记录

通知列表页面

如图4 智慧校园通知界面，用户可以看到自己待查看的消息，十分详细简介。



图 4 用户通知列表界面

通知详情页面

如图5 通知详情界面，用户可以查看到详细的通知详情。包括发布的部门，时间，内容，注意事项等等内容。



图 5 用户通知详情界面

已读状态效果

如图 6 已读状态效果图所示用户可以看到自己已读的消息变成了阴影状态取消了红点的标注，让用户知道了自己已经查看了这条消息。还会保留在消息列表，用户想查看还能继续查看。



图 6 已读状态效果图

问题与解决

问题描述	解决方案	解决结果
首次使用微信开发者工具不熟悉界面	查看官方文档和教程，逐步熟悉各功能模块	已解决，掌握了基本开发流程
WXSS样式不生效	检查类名和选择器，确保与WXML一致	已解决，样式正常显示
本地存储的数据在刷新后丢失	检查wx.setStorageSync和wx.getStorageSync的使用	已解决，数据持久化正常
详情页接收不到参数	检查wx.navigateTo的url参数传递格式	已解决，参数传递正常

实验总结

实验收获

- 掌握了微信小程序的完整开发流程，从项目创建到真机预览
- 学会了WXML、WXSS、JavaScript三种核心语言的使用
- 理解了小程序的页面路由和数据传递机制
- 掌握了本地存储wx.setStorageSync和wx.getStorageSync的使用
- 体验了AI辅助编程工具TRAE在代码生成和调试中的作用

技术要点

- 使用wx:for进行列表渲染，wx:key提升渲染性能
- 通过bindtap绑定点击事件，data-*传递自定义数据
- 使用wx.navigateTo进行页面跳转，options获取参数
- 使用wx.setStorageSync和wx.getStorageSync实现本地数据持久化
- 启用enablePullDownRefresh实现下拉刷新功能
- 通过条件渲染class="{{item.isRead ? 'read' : ''}}"实现已读状态视觉反馈

考核指标

考核项	评分标准	得分
功能完整性	通知列表、详情页、本地存储功能完整	30分
代码质量	代码结构清晰，注释完整	20分
页面设计	界面美观，用户体验良好	15分
本地存储	已读状态存储正常	15分
真机预览	小程序可在真机上正常运行	10分
实验报告	报告结构完整，内容详实	10分

教师评价

项目	评价
实验完成情况	实验任务全部完成，功能完善
技术掌握程度	掌握了小程序开发的核心技术
创新点	使用了AI辅助编程，提升了开发效率
综合评价	优秀
成绩	

教师签名： _____

日期： _____